

在本题中，你不必追求很高的效率，而需要在意功能的实现。所有的数据点均下发。通过非常规手段通过测试点，将不会给分。

fix: 你应当保证 NFA 的 transitions 的大小，与 NFA 中的节点数相同。

我们有一串由月与花束组成的序列。现在，花谱想观测这一串序列是否符合一定的模式，请你帮她解决这个问题。

为了将题目进一步抽象化，我们用字符 `a` 来代表月亮，字符 `b` 来代表花束。也就是说，我们会给出一个由 `a b` 组成的字符串，希望你判断这些字符串是否符合一定的模式。具体而言，除去字符之外，我们有这些符号：

- `+`. 如 `a+` 表示可以匹配一次或多次 `a`.
- `*`. 如 `b*` 表示可以匹配零次或多次 `b`.
- `?`. 如 `a?` 表示可以匹配零次或一次 `a`.
- 字符串之间可以连接。如 `ab+a*` 代表先匹配 `a`，再匹配 `b+`，再匹配 `a*`.
- `|`. 如 `a+b|ba*` 代表既可以匹配 `a+b`，也可以匹配 `ba*`. 该运算的优先级是最低的。

也就是说，我们的表达式中不会有括号。那么，我们应该如何解决这个问题呢？玉川在这里给出一些简单的介绍。我们可以使用一种被称为 **非确定性有限状态自动机(NFA)** 的方式来解决。为了便于理解，我们会先给出一个形式化的定义，然后给出一个例子来说明。

(为了避免歧义，我们用大写英文字母代表集合，小写英文字母代表元素) 形式化地说，NFA 就是一个五元组 $(\Sigma \cup \{\epsilon\}, F, S, \delta(s, a), I)$. 其中， S 代表 NFA 上的所有的状态， $\delta(s, a)$ 代表着一个迁移函数，即“若在 s 状态下，接收到一个字母 a , 则可以转移到 ...”的一个函数。在每一个时刻，我们可以认为 NFA 处在一组状态 S' , 即 $S' \subset S$ 上。在每一个时刻，NFA 可以接收一个字母 a ，然后转移到一个新的状态组 $S'' = \bigcup_{s \in S'} \delta(s, a)$ 上。另外，在 NFA 中，我们允许空转移 ϵ 的存在。所以，在进行上述操作之前，我们需要找出当前状态集，可以仅通过 ϵ 转移到的状态的集合，然后再执行上述操作，并且再一次找出新的可以仅通过 ϵ 转移到的状态集合。此外， Σ 就是字母表，即代表着所有可能出现的字母的集合。即在 F 代表着终止状态。即，若 NFA 接收了所有字符后，停在了一个状态集 D 上，若 $D \cap F \neq \emptyset$, 那么我们就称这个字符串被 NFA 接受。 I 代表着初始状态集合，即在未接收任何字符时，我们所处在的状态集上(在本题中，初始状态集总可以是大小为 1 情况)。实际上，你要做的，就是 **实现一个根据简单模式串，构造一个只能够接受要求字符串的 NFA**。

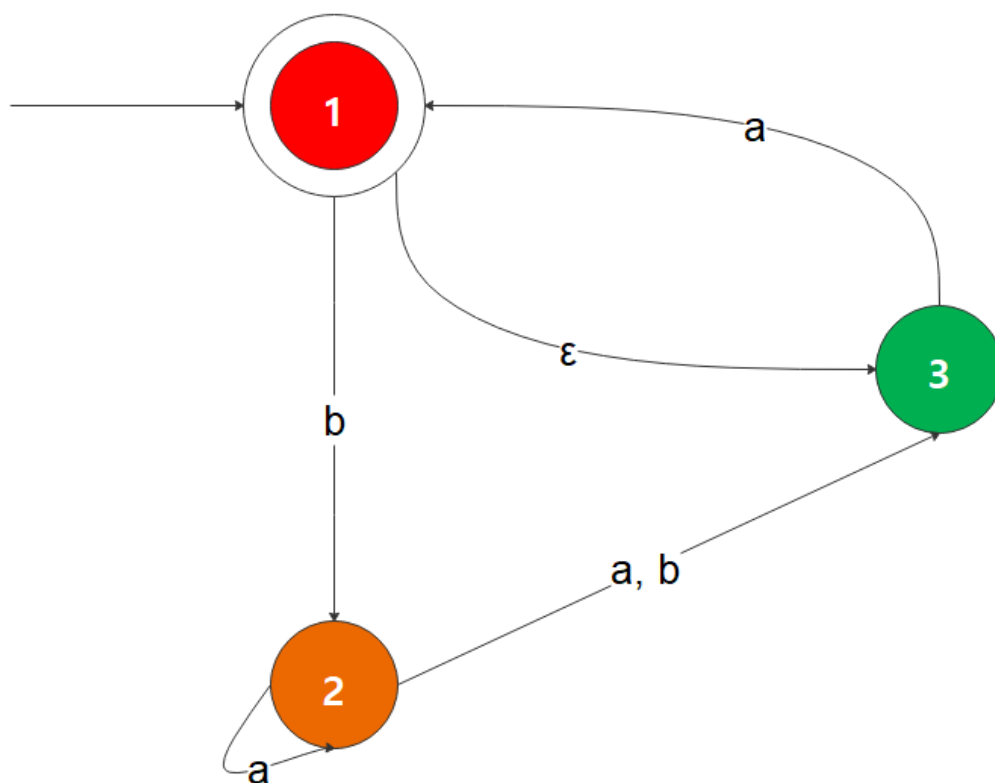
此外，为了减轻大家对特殊情况的工作量，我们规定空字符串是 **未定义** 的测试样例，也就是说，你不必考虑检查空字符串是否符合某个模式。

那么，我们的工作流就变得明晰了：

- 首先，根据给出的模式字符串，构建出 NFA.
- 在构建出的 NFA 上，检查这个状态是否被接受。

在 `Advance(S, a)` (给定状态集 S 和字符 a , 返回迁移后的状态集 S' 的函数) 过程中，我们可以更加细致地描述为：

- 给出一组当前状态，记作 S .
- 检查在这些状态中，可以只通过 ϵ 边可以到达哪些状态，记作 $\bar{S}$. 为了方便你的工作，玉川已经给出了一个函数，可以获得一个状态通过 ϵ 边到达的所有状态（包括它自己）。
- 从 $\bar{S}$ 中的状态出发，经过和当前输入的字符相同的转移边，记到达的所有状态集合记作 D .
- 检查所有 D 中的状态，可以只通过 ϵ 边到达哪些状态，记作 $\bar{D}$. 返回 $S' = \bar{D}$.



在这里，我们给出一个 NFA 的例子。如上图。这个 NFA 的初始状态是 1，终止状态也为 1. 这个 NFA 可以接受字符串 `aaa`，因为它可以通过 ϵ 边到达状态 3，然后通过 `a` 边回到状态 1，如此往复三次。它也可以接受字符串 `baaba`，通过状态 1,2,2,3,1 这样的转移顺序。此外，玉川决定再给出一点小提示。对于模式串 `a*` 而言，我们可以构造这样的 NFA:

- 状态为 0.
- 初始状态为 0.

- 终止状态为 0.
- 有一条转移边, 为 $0 \rightarrow a \rightarrow 0$,

在本题中, 玉川已经为你实现了一个 `NFA` 类, 请你完善剩下的功能。 下发测试点在[这里](#)

代码模版

```

#pragma once
#include <queue>
#include <string>
#include <unordered_set>
#include <vector>
// You are NOT allowed to add any headers
// without the permission of TAs.
namespace Grammar {
class NFA;
NFA MakeStar(const char &character);
NFA MakePlus(const char &character);
NFA MakeQuestion(const char &character);
NFA Concatenate(const NFA &nfa1, const NFA &nfa2);
NFA Union(const NFA &nfa1, const NFA &nfa2);
NFA MakeSimple(const char &character);
/*!
    \brief This class is used to store the type of the transition.
    \details You can use it like this:
    \code
        TransitionType type = TransitionType::Epsilon;
        TransitionType type2 = TransitionType::a;
        TransitionType type3 = TransitionType::b;
    \endcode
*/
enum class TransitionType { Epsilon, a, b };
struct Transition {
    /*!
        \brief value is used to store which character to match.
    */
    TransitionType type;
    /*!
        \brief This is used to store which node the transition is going to.
    */
    int to;
    Transition(TransitionType type, int to) : type(type), to(to) {}
};
class NFA {
private:
    /*!
        \brief This is used to store the start state of the NFA.
    */
    int start;
    /*!
        \brief This is used to store the end state of the NFA.
    */
    std::unordered_set<int> ends;
    /*!
        \brief This is used to store the transitions of the NFA.
        \details For example, transitions[3] stores all the transitions beginning
        with \details state 3.
    */

```

```

*/
std::vector<std::vector<Transition>> transitions;

public:
    NFA() = default;
    ~NFA() = default;
    /*!
        \brief Get the epsilon closure of a state.
        \param state The state to get the epsilon closure of.
        \return The epsilon closure of the state.
    */
    std::unordered_set<int>
    GetEpsilonClosure(std::unordered_set<int> states) const {
        std::unordered_set<int> closure;
        std::queue<int> queue;
        for (const auto &state : states) {
            if (closure.find(state) != closure.end())
                continue;
            queue.push(state);
            closure.insert(state);
        }
        while (!queue.empty()) {
            int current = queue.front();
            queue.pop();
            for (const auto &transition : transitions[current]) {
                if (transition.type == TransitionType::Epsilon) {
                    if (closure.find(transition.to) == closure.end()) {
                        queue.push(transition.to);
                        closure.insert(transition.to);
                    }
                }
            }
        }
        return closure;
    }
    /*!
        \brief Advance the NFA to the next states.
        \param current_states The current states of the NFA.
        \param character The character to match.
        \return The next states of the NFA.
    */
    std::unordered_set<int> Advance(std::unordered_set<int> current_states,
                                    char character) const {
        // TODO:
    }

    /*!
        \brief Check if a state is accepted.
        \param state The state to check.
        \return true if the state is accepted, false otherwise.
    */

```

```

bool IsAccepted(int state) const { return ends.find(state) != ends.end(); }

/*!
 \brief Return the start state.
 \return The start state.
*/

int GetStart() const { return start; }

friend NFA MakeStar(const char &character);
friend NFA MakePlus(const char &character);
friend NFA MakeQuestion(const char &character);
friend NFA MakeSimple(const char &character);
friend NFA Concatenate(const NFA &nfa1, const NFA &nfa2);
friend NFA Union(const NFA &nfa1, const NFA &nfa2);
};

class RegexChecker {
private:
    /*!
     \brief This is used to store the regex string.
    */
    NFA nfa;

public:
    /*!
     \brief To check if the string is accepted by the regex.
     \param str The string to be checked.
     \return true if the string is accepted by the regex, false otherwise.
    */
    bool Check(const std::string &str) const {
        // TODO:
    }

    /*!
     \brief This is used to build the NFA from the regex string.
    */
    RegexChecker(const std::string &regex) {
        // TODO:
    }
};

/*!
 \brief Return a NFA to match a* or b*.
 \param character The character to match.
 \details This function will create a NFA that matches a* or b*.
*/
NFA MakeStar(const char &character) {
    NFA nfa;
    nfa.start = 0;
    nfa.ends.insert(0);
    nfa.transitions.push_back(std::vector<Transition>());
    if (character == 'a') {

```

```

    nfa.transitions[0].push_back({TransitionType::a, 0});
} else {
    nfa.transitions[0].push_back({TransitionType::b, 0});
}
return nfa;
}
/*!
 \brief Return a NFA to match a+ or b+.
 \param character The character to match.
 \details This function will create a NFA that matches a+ or b+.
*/
NFA MakePlus(const char &character) {
    // TODO:
}
/*!
 \brief Return a NFA to match a? or b?.
 \param character The character to match.
 \details This function will create a NFA that matches a? or b?.
*/
NFA MakeQuestion(const char &character) {
    // TODO:
}

/*!
 \brief Return a NFA to match ...(the first part) ...(the second part).
 \param nfa1 The first NFA to match, for example, it is a.
 \param nfa2 The second NFA to match, for example, it is b.
 \details This function will create a NFA that matches ab.
*/
NFA Concatenate(const NFA &nfa1, const NFA &nfa2) {
    // TODO:
}
/*!
 \brief Return a NFA to match ... | ...
 \param nfa1 The first NFA to match, for example, it is a.
 \param nfa2 The second NFA to match, for example, it is b.
 \details This function will create a NFA that matches a|b.
*/
NFA Union(const NFA &nfa1, const NFA &nfa2) {
    // TODO:
}

/*!
 \brief Return a NFA to match a, or return a NFA to match b.
 \param character The character to match.
 \return The required NFA.
*/
NFA MakeSimple(const char &character) {
    // TODO:
}
} // namespace Grammar

```

分数构成

Testcase No.	考察内容	分值
1	`MakeSimple()` 和 `Advance()`	15
2	`Concatenate()` 和 `Advance()`	15
3	`MakePlus()` 和 `Advance()`	15
4	`MakeQuestion()` 和 `Advance()`	15
5	`MakeStar()` 下的 和 `Advance()`	5
6	`Union()` 和 `Advance()`	15
7	`Regchecker` 的构造函数 和 `Check()` 函数	20